❹ Node 1 accepts node 2

❺ Node 2 advertises its Up state

The output of node 2 is very similar to that of node 1, except that from the moment it is discovered by node 1 until the time node 1 forms the seed node, it loops only over empty seed nodes and fewer contact points than required. Node 2 is informed by the configuration that it does not have the lowest IP, so it doesn't try to discover whether the number of contact points is sufficient. It waits until it finds a seed node via the Akka Management HTTP endpoint, and it also uses the endpoint to send an `InitJoin`. The following output of node 2 shows the final cluster greeting by node 1, indicating that it is considered `Up`:

```
Cluster Node [akka://testing-bootstrap@127.0.0.2:25520] - Welcome from
➥[akka://testing-bootstrap@127.0.0.1:25520]
ClusterReceptionist [akka://testing-bootstrap@127.0.0.2:25520] - Node added
➥[UniqueAddress(akka://testing-bootstrap@127.0.0.1:25520,-
➥2336789632363653010)]
```

You've seen how to start a two-node cluster using Akka Bootstrap. Now let's look at how to bootstrap the cluster in Kubernetes.

13.2 Clustering with the Kubernetes API

Instead of using fixed IPs, you can use the most common solution: the Kubernetes API. To begin, you need to pack the application into a container. Usually, you create a Docker container using `sbt-native-packager`. To do this, import its plugin by adding it to the plugins.sbt file: